Estructura de datos común · Perfil del estudiante ↔ Oferta de trabajo ↔ Match

> **Estado:** borrador tentativo (v0.1.0-draft). No hay código que lo implemente todavía. **Base:** `cv_standard.schema.json` (extracción de CV por IA, sin `null`). **Uso:** vocabulario compartido entre `profile-service`, `jobs-service`, `matching-service`, `mcp-tools`, `n8n` y `frontend`. Antes de implementar, discutir en la daily y, cuando se apruebe, convertirlo en JSON Schemas en `contracts/` (ver §9).

---

1. Idea central

El perfil del estudiante y la oferta de trabajo se describen **con los mismos bloques**. Lo que cambia es cómo se lee cada uno:

Bloque	En el **perfil** significa…	En la **oferta** significa…	El **match** hace…
`location`	Dónde vive el estudiante	Dónde está el puesto	Calcula la distancia o compatibilidad geográfica
`work_conditions`	Lo que **busca o acepta**	Lo que **ofrece**	Cruza los conjuntos (intersección)
`work_conditions.salary`	**Sueldo pretendido**	**Sueldo ofrecido**	Compara rangos y calcula la brecha (§7.2)
`skills[]`	Lo que **tiene** (con nivel)	Lo que **pide** (con nivel y tipo `required/desired/excluding`)	Score por reglas y skills faltantes
`languages[]`	Los idiomas que habla	Los idiomas que pide	Criterio Languages
`education[]`	Su formación	La formación mínima que pide	Criterio Education
`experience[]`	Su experiencia	Años mínimos y área	Criterio Experience
`certifications[]`	Las que tiene	Las que pide o valora	Criterio opcional

Así un mismo normalizador (minúsculas, sin acentos, sinónimos) y los mismos enums sirven para los tres lados.

---

2. Reglas transversales

1. **Claves en inglés y `snake_case`**, igual que `cv_standard.schema.json`. Los valores visibles se traducen en el frontend.
2. **Sin `null`** en datos que vienen de la IA: `"COMPLETAR_MANUALMENTE"` en escalares, `[]` en listas (regla de `cv_standard`). En datos que confirmó el usuario se permite omitir campos opcionales.
3. **Enums compartidos** (§3): se definen una sola vez y se reutilizan (`$ref`) en los tres schemas.
4. **Datos sensibles** (AGENTS.md §5.5): no se guarda la dirección exacta (solo ciudad, provincia y opcionalmente código postal). Los datos de inclusión o salud son opcionales, los carga sólo el usuario y **nunca** entran a prompts de IA ni a las salidas.
5. **Origen**: todo lo calculado lleva `origin: "RULES"` + `engine_version`; todo lo generado por IA lleva `origin: "AI"` + `model`, `generated_at`, `prompt_version`, `disclaimer`.

6. **Trazabilidad del dato** en perfil: cada ítem puede llevar `source: "cv_ai" | "manual"` y `confirmed: bool`, para distinguir sugerencias de IA de lo que aceptó el usuario.

---

3. Catálogos / enums compartidos

Enum	Valores	Notas
`work_schedule` (modalidad horaria)	`full_time`, `part_time`, `hourly`, `flexible`	`hourly` = por horas o por turnos sueltos
`work_modality` (presencialidad)	`onsite`, `remote`, `hybrid`	presencial, remota o mixta
`employment_type` (tipo de vínculo)	`employee`, `freelance`, `internship`, `trainee`	`employee` = relación de dependencia; `internship` = pasantía (Ley 26.427)
`contract_duration`	`permanent`, `fixed_term`, `project`, `seasonal`	indeterminado, plazo fijo, por proyecto, temporada
`availability_start` (disponibilidad de inicio)	`immediate`, `within_15_days`, `within_30_days`, `to_agree`	
`shift` (franja horaria)	`morning`, `afternoon`, `evening`, `night`, `weekend`, `rotating`	Sirve para estudiantes que cursan en un turno
`seniority`	`no_experience`, `trainee`, `junior`, `semi_senior`, `senior`	
`skill_level`	`1` – `5`	Ya usado en `cv_standard` y en Profile
`skill_category`	`hard`, `soft`, `tool`, `methodology`	Opcional, para filtrar y presentar
`requirement_kind`	`required`, `desired`, `excluding`	Sólo en la oferta (ya definido en jobs-service)
`language_level`	`A1`, `A2`, `B1`, `B2`, `C1`, `C2`, `native`	MCER; la IA puede devolver texto libre y se normaliza
`education_level`	`secondary`, `tertiary`, `university`, `postgraduate`, `course`	
`education_status`	`in_progress`, `completed`, `incomplete`	
`currency`	`ARS`, `USD`	
`salary_period`	`hour`, `day`, `month`, `year`, `project`	Para comparar se normaliza a `month` (§7.2)
`salary_visibility`	`public`, `on_match`, `hidden`	Pública, visible sólo a la otra parte cuando hay match o postulación, u oculta (sólo la usa el motor)
`salary_adjustment`	`none`, `monthly`, `quarterly`, `semiannual`, `by_agreement`	Ajuste por inflación (sobre todo en ARS)
`payment_currency_mode`	`local`, `usd`, `usd_paid_in_local`	Cobro en pesos, en dólares o en dólares liquidados en pesos
`job_status`	`draft`, `active`, `paused`, `closed`, `expired`	Hoy jobs-service usa `active` / `closed`
`application_status`	`suggested`, `applied`, `viewed`, `shortlisted`, `interview`, `offer`, `hired`, `rejected`, `withdrawn`	Estado del match como postulación
`profile_visibility`	`public_to_companies`, `only_applied`, `hidden`	

---

4. Bloques comunes (tipos reutilizables)

4.1 `Location`

Campo	Tipo	Req.	Descripción
`country`	string (ISO 3166-1 alfa-2, ej. `AR`)	sí	
`province`	string	sí	Provincia o estado
`city`	string	sí	Ciudad o localidad
`postal_code`	string	no	Sin calle ni número
`zone`	string	no	Barrio o zona aproximada (ej. "Zona Norte GBA")

4.2 `SalaryRange` — sueldo pretendido / ofrecido

Es el mismo tipo en los dos lados, guardado en `work_conditions.salary`: - **Perfil** → **sueldo pretendido**: lo mínimo que acepta el estudiante (`min`) y, si quiere, a qué aspira (`max`). - **Oferta** → **sueldo ofrecido**: el rango que paga la empresa por el puesto.

Campo	Tipo	Req.	Perfil (pretendido)	Oferta (ofrecido)
`min`	number ≥ 0	sí si hay monto	Piso aceptable	Mínimo del rango
`max`	number ≥ `min`	no	Aspiración	Máximo del rango
`currency`	`currency`	sí si hay monto		
`period`	`salary_period`	sí si hay monto	ej. `hour` para freelance	
`gross`	bool	no (default `true`)	Bruto o neto	Bruto o neto
`payment_mode`	`payment_currency_mode`	no	Cómo quiere cobrar	Cómo se paga
`negotiable`	bool	no	Acepta negociar por debajo del piso	El rango se puede negociar
`adjustment`	`salary_adjustment`	no	—	Política de ajuste
`variable_compensation`	`{type: bonus|commission|profit_sharing, description, max_percent}` []	no	—	Remuneración variable
`visibility`	`salary_visibility`	no (default `on_match`)	Quién ve su pretensión	Si se publica el rango en el aviso
`to_agree`	bool	no	"A convenir": no carga monto	"A convenir": no publica monto
`notes`	string	no		ej. "más asignación por conectividad"

Reglas: - Si `to_agree = true`, `min` / `max` pueden faltar y el chequeo salarial da `unknown` (no penaliza). - Para pasantías, el sueldo ofrecido es la **asignación estímulo** (`internship.allowance` usa el mismo tipo). - El monto nunca se manda a los prompts de IA si `visibility = hidden`.

4.3 `WorkConditions` (lo que se busca ↔ lo que se ofrece)

En el perfil cada lista significa "acepto cualquiera de estas"; en la oferta, "el puesto es de este tipo" (normalmente un solo valor, aunque puede haber más).

Campo	Tipo	Descripción
`work_schedule`	`work_schedule[]`	full time / part time / por horas / flexible
`work_modality`	`work_modality[]`	presencial / remota / mixta
`hybrid_onsite_days`	int (0–5)	Días presenciales por semana si es `hybrid` (en el perfil = máximo que acepta)
`employment_type`	`employment_type[]`	relación de dependencia / freelance / pasantía / trainee
`contract_duration`	`contract_duration[]`	
`weekly_hours`	`{min:int, max:int}`	Horas semanales (perfil: disponibles; oferta: requeridas)
`shifts`	`shift[]`	Franjas disponibles o requeridas
`availability_start`	`availability_start`	Perfil: cuándo puede empezar. Oferta: cuándo se necesita
`salary`	`SalaryRange` (§4.2)	Perfil: **sueldo pretendido**. Oferta: **sueldo ofrecido**
`willing_to_relocate`	bool	Sólo perfil
`relocation_support`	bool	Sólo oferta
`max_commute_km`	int	Sólo perfil: distancia máxima aceptada para presencial o mixta
`travel_availability`	`none` | `occasional` | `frequent`	Perfil: disponibilidad para viajar. Oferta: viajes que exige el puesto
`driver_license`	string[] (ej. `["B1"]`)	Perfil: las que tiene. Oferta: las que requiere
`own_vehicle`	bool	Igual criterio
`own_equipment`	bool	Para remoto o freelance: equipo propio (perfil) o requerido (oferta)
`invoicing_status`	`monotributo` | `responsable_inscripto` | `none` | `to_register`	Relevante para freelance (perfil: su situación; oferta: lo que exige)
`work_authorization`	string[] (países ISO)	Perfil: países donde puede trabajar. Oferta: países admitidos (remoto internacional)
`timezone`	string IANA (ej. `America/Argentina/Buenos_Aires`)	Para remoto
`timezone_overlap_hours`	int	Oferta: horas mínimas de solapamiento horario

4.4 `SkillItem`

Extiende el ítem de `cv_standard.skills[]` (`name`, `level`).

Campo	Tipo	Perfil	Oferta	Descripción
`name`	string	✓	✓	Como aparece
`normalized`	string	✓	✓	Clave del catálogo (`skills_catalog.json`) tras sinónimos

Campo	Tipo	Perfil	Oferta	Descripción
`level`	`skill_level`	✓ (tiene)	✓ (mínimo pedido)	
`years`	number	opc.	opc. (mínimo)	Años de uso
`category`	`skill_category`	opc.	opc.	
`kind`	`requirement_kind`	—	✓	required / desired / excluding
`weight`	number	—	opc.	Peso relativo dentro de su tipo (default 1)
`last_used`	string `YYYY`	opc.	—	
`source` / `confirmed`	ver §2.6	✓	—	

4.5 `LanguageItem`

Campo	Tipo	Perfil	Oferta
`name` / `normalized`	string	✓	✓
`level`	`language_level`	tiene	mínimo pedido
`kind`	`requirement_kind`	—	✓
`certification`	string	opc. (ej. "FCE")	opc.

4.6 `EducationItem`

Extiende `cv_standard.education[]` (`institution`, `degree`, `status`, `start`, `end`).

Campo	Tipo	Perfil	Oferta
`institution`	string	✓	opc. (si se prefiere alguna)
`degree`	string	✓	✓ (título o carrera afín)
`field`	string	opc. (área: sistemas, administración…)	✓
`level`	`education_level`	✓	✓ (mínimo)
`status`	`education_status`	✓	mínimo aceptado (ej. `in_progress` = "estudiante avanzado")
`start` / `end`	string `YYYY-MM`	✓	—
`progress_percent`	int 0–100	opc. (materias aprobadas)	mínimo pedido
`current_year`	int	opc.	—
`gpa`	number	opc.	opc. (mínimo)
`kind`	`requirement_kind`	—	✓

4.7 `ExperienceItem`

Extiende `cv_standard.experience[]` (`company`, `role`, `start`, `end`, `description`).

Campo	Tipo	Descripción
`company`, `role`, `start`, `end`, `description`	string	Igual a `cv_standard`; `end` = `"actual"` si sigue
`area`	string	Área o industria (normalizada)
`employment_type`	`employment_type`	Dependencia, freelance, pasantía…
`work_modality`	`work_modality`	

Campo	Tipo	Descripción
`skills_used`	string[] (normalizadas)	Permite calcular `years` por skill
`months`	int	Calculado por reglas a partir de `start` / `end`

En la oferta, la experiencia se pide de forma agregada: `experience_requirement = { min_years, area, kind }`.

---

5. `StudentProfile` (profile-service)

Es `cv_standard` confirmado por el usuario, con los bloques de §4 y datos propios de la búsqueda.

```
StudentProfile
├── id, user_id, created_at, updated_at, version
├── personal                        (de cv_standard)
│   ├── name, email, phone
│   ├── birth_year                  (opc.; sólo para validar pasantías, no se usa en matching)
│   └── location: Location          (reemplaza al string "location" de cv_standard)
├── summary                         (de cv_standard)
├── headline                        ej. "Estudiante de Ing. en Sistemas – Desarrollo web"
├── academic_status                 (específico de bolsa estudiantil)
│   ├── institution, career, current_year, progress_percent
│   ├── expected_graduation         YYYY-MM
│   └── student_shift               shift[]  (turno en que cursa → evita choques)
├── target
│   ├── roles[]                     puestos buscados ("Desarrollador frontend", "Analista QA")
│   ├── areas[]                     industrias o áreas de interés
│   └── seniority                   seniority
├── work_conditions: WorkConditions  (preferencias: lo que busca o acepta)
│   └── salary: SalaryRange         SUELDO PRETENDIDO (piso, aspiración, moneda, período, visibilidad)
├── skills[]: SkillItem
├── experience[]: ExperienceItem
├── education[]: EducationItem
├── languages[]: LanguageItem
├── certifications[]                {name, issuer, year, url}  (cv_standard las tiene como string[])
├── projects[]                      {name, description, url, skills_used[]}  (portfolio, clave para estudiantes sin
experiencia)
├── links[]                         {type: linkedin|github|portfolio|other, url}
├── volunteering[]                  {organization, role, start, end, description}
├── references_available            bool
├── visibility                      profile_visibility
├── job_search_status               active | open_to_offers | not_looking
├── consents                        {data_processing: bool, ai_processing: bool, contact_by_email: bool,
accepted_at}
├── inclusion (opc., privado)       {disability_certificate: bool, accessibility_needs: string}  → nunca a IA
├── cv_files[]                      {stored_filename, mime_type, uploaded_at}
└── cv_suggestions                  (salida de WF-01, origin="AI"; no pisa lo confirmado)
```

---

6. `JobOpportunity` (jobs-service)

```
JobOpportunity
├── id, company_id, created_at, updated_at, version
├── status                          job_status
├── published_at, expires_at
├── title                           ej. "Desarrollador/a Frontend Trainee"
├── description                     texto libre
├── responsibilities[]              string[]
├── area                            área o industria
├── seniority                       seniority
├── vacancies                       int
├── location: Location              (sede; en remoto puro puede ser la de la empresa)
├── work_conditions: WorkConditions  (lo que ofrece el puesto)
│   └── salary: SalaryRange         SUELDO OFRECIDO (rango, variable, ajuste, si se publica)
├── requirements
```

```
│   ├── skills[]: SkillItem       (con kind; ≥ 1 required, regla vigente)
│   ├── languages[]: LanguageItem
│   ├── education[]: EducationItem
│   ├── experience: {min_years, area, kind}
│   ├── certifications[]          {name, kind}
│   └── academic                  {min_progress_percent, careers[], min_current_year}  (opc.)
├── benefits[]                    ej. "prepaga", "capacitación", "bono", "horario flexible", "días de estudio"
├── internship (si employment_type incluye internship)
│   ├── institution_agreement     bool (convenio con la universidad)
│   ├── allowance                 SalaryRange (asignación estímulo)
│   └── duration_months           int
├── application
│   ├── deadline                  YYYY-MM-DD
│   ├── screening_questions[]     {id, text, type: text|yes_no|choice, required}
│   └── contact_visible           bool
├── company_snapshot              {name, industry, size: 1-10|11-50|51-200|201-1000|1000+, logo_url}
│                                 (copia de lectura; la fuente es auth-service.Company)
└── inclusive_hiring (opc.)       {accepts_disability_certificate: bool, accessible_workplace: bool}
```

---

7. `Match` (matching-service)

Une **un perfil** y **una oferta**. Mantiene separados el bloque calculado por reglas y el generado por IA.

```
Match
├── id
├── profile_id, job_id
├── profile_version, job_version      (para saber si el cálculo quedó viejo)
├── computed_at
├── rule_based                        origin="RULES"
│   ├── engine_version, rules_version (hash o versión de scoring_rules.yaml)
│   ├── score                         0–100 (criterios actuales; tope 40 si falta un excluding)
│   ├── excluding_cap_applied         bool
│   ├── breakdown[]                   {criterion, weight, earned, details}
│   │                                 RequiredSkills 50 · DesiredSkills 15 · Levels 15 ·
│   │                                 Experience 10 · Education 5 · Languages 5
│   ├── missing_skills[]              {skill, kind, required_level, current_level, gap, priority}
│   ├── conditions_fit                (NUEVO, ver §7.1)
│   │   ├── compatible                bool (pasa todos los filtros duros)
│   │   ├── checks[]                  {condition, result: match|partial|mismatch|unknown, profile_value,
job_value, hard: bool}
│   │   ├── distance_km               number | "COMPLETAR_MANUALMENTE"
│   │   └── salary_fit                (ver §7.2)
│   │       ├── result                match | partial | below_expectation | unknown
│   │       ├── expected_monthly      {min, max} pretendido normalizado a mensual bruto
│   │       ├── offered_monthly       {min, max} ofrecido normalizado a mensual bruto
│   │       ├── gap_percent           (piso pretendido − máx. ofrecido) / piso pretendido × 100; 0 si se solapan
│   │       └── currency              moneda en la que se comparó
│   └── matched_skills[]              {skill, profile_level, required_level, kind}
├── ai_recommendations                origin="AI" (WF-06; student_recommendations / company_recommendations)
│   ├── model, generated_at, prompt_version, disclaimer
│   └── items[]                       máx. 5, priority alta|media|baja
├── application                       (cuando el match pasa a postulación)
│   ├── status                        application_status
│   ├── applied_at, updated_at
│   ├── screening_answers[]           {question_id, answer}
│   ├── cover_letter                  string
│   └── history[]                     {status, at, by: student|company|system}
├── feedback (opc.)                   {by: student|company, useful: bool, reason}
└── notifications[]                   {notification_key, channel, sent_at}   (idempotencia WF-02/03)
```

7.1 Condiciones laborales en el match (propuesta)

Para **no romper los pesos actuales** (que suman 100), las condiciones laborales van en un bloque separado `conditions_fit`, también calculado por reglas. Cada chequeo es una clase `ConditionCheck`, igual que

`ScoringCriterion` (OCP). Cada una se configura en `scoring_rules.yaml` como **filtro duro** (`hard: true`, si no coincide se descarta o se marca incompatible) o **blando** (sólo informa).

Condición	Perfil	Oferta	Regla sugerida	Duro por defecto
Presencialidad	`work_modality[]`	`work_modality[]`	Intersección no vacía	sí
Ubicación / traslado	`location`, `max_commute_km`, `willing_to_relocate`	`location`, `relocation_support`	Si es onsite/hybrid: misma ciudad, distancia ≤ max, o acepta mudarse	sí
Modalidad horaria	`work_schedule[]`, `weekly_hours`	`work_schedule[]`, `weekly_hours`	Intersección y rangos de horas que se solapan	sí
Franja horaria	`shifts[]`, `academic_status.student_shift`	`shifts[]`	Los turnos del puesto no chocan con los de cursada	no
Tipo de vínculo	`employment_type[]`	`employment_type[]`	Intersección no vacía	sí
Duración	`contract_duration[]`	`contract_duration[]`	Intersección	no
Disponibilidad de inicio	`availability_start`	`availability_start`	Perfil ≤ oferta	no
Sueldo pretendido / ofrecido	`salary` (pretendido)	`salary` (ofrecido)	Ver §7.2	no
Viajes	`travel_availability`	`travel_availability`	Perfil ≥ oferta	no
Licencia / vehículo	`driver_license`, `own_vehicle`	ídem	Incluye lo requerido	sí si la oferta lo exige
Autorización / zona horaria	`work_authorization`, `timezone`	ídem + `timezone_overlap_hours`	Contiene el país y alcanza el solapamiento	sí (remoto internacional)
Facturación (freelance)	`invoicing_status`	`invoicing_status`	Igual o `to_register`	no
Académico	`academic_status`	`requirements.academic`	progress/year ≥ mínimo, carrera en la lista	configurable

Si falta el dato en alguno de los dos lados, el resultado es `unknown` (no penaliza; el frontend sugiere completar el perfil).

7.2 Comparación sueldo pretendido ↔ ofrecido (`SalaryCheck`)

1. **Normalizar** los dos lados a mensual y bruto: `hour × weekly_hours × 4.33`, `day × 21`, `year ÷ 13` (12 sueldos + aguinaldo), `project` no se convierte (da `unknown`). Para neto → bruto se usa un factor configurable en `scoring_rules.yaml` (ej. `net_to_gross: 1.21`).
2. **Moneda**: si difieren, se compara sólo si hay un tipo de cambio configurado (`fx_rates` en `scoring_rules.yaml`, con fecha); si no, `unknown`.
3. **Resultado**: - `match`: el rango ofrecido llega al piso pretendido (`offered.max ≥ expected.min`). - `partial`: no llega, pero la brecha es ≤ `salary_tolerance_percent` (ej. 10 %) o alguna de las partes marcó `negotiable`. - `below_expectation`: la brecha supera la tolerancia. - `unknown`: falta el dato, `to_agree`, `project` o monedas sin tipo de cambio.
4. **Privacidad**: el motor siempre compara, pero la respuesta a cada parte respeta `visibility`. Si la pretensión del estudiante es `hidden`, la empresa ve sólo `result` y no los montos; lo mismo al revés.

5. No modifica el score (regla de §7.1); sirve para filtrar, ordenar y avisar ("el sueldo ofrecido está 15 % debajo de tu pretensión").

Alternativa a decidir: sumar un criterio `WorkConditions` al score (por ejemplo, 10 puntos, reescalando los demás). Rompe la comparabilidad con la versión actual del motor y exige un MAJOR en `engine_version`.

---

8. Ejemplo mínimo (JSON)

Perfil (extracto):

```
{
  "personal": {
    "name": "Lucía Pérez",
    "email": "lucia.perez@example.com",
    "phone": "COMPLETAR_MANUALMENTE",
    "location": { "country": "AR", "province": "Buenos Aires", "city": "San Isidro" }
  },
  "academic_status": { "institution": "UTN FRBA", "career": "Ing. en Sistemas", "current_year": 4,
"progress_percent": 70, "student_shift": ["evening"] },
  "work_conditions": {
    "work_schedule": ["part_time"],
    "work_modality": ["hybrid", "remote"],
    "hybrid_onsite_days": 2,
    "employment_type": ["employee", "internship"],
    "weekly_hours": { "min": 20, "max": 30 },
    "shifts": ["morning", "afternoon"],
    "availability_start": "immediate",
    "max_commute_km": 25,
    "willing_to_relocate": false,
    "salary": {
      "min": 600000,
      "max": 750000,
      "currency": "ARS",
      "period": "month",
      "gross": true,
      "negotiable": true,
      "visibility": "on_match",
      "to_agree": false
    }
  },
  "skills": [
    { "name": "React", "normalized": "react", "level": 3, "years": 1, "source": "cv_ai", "confirmed": true },
    { "name": "Git", "normalized": "git", "level": 3, "source": "manual", "confirmed": true }
  ],
  "languages": [{ "name": "Inglés", "normalized": "ingles", "level": "B2" }]
}
```

Oferta (extracto):

```
{
  "title": "Desarrollador/a Frontend Trainee",
  "status": "active",
  "seniority": "trainee",
  "location": { "country": "AR", "province": "CABA", "city": "Buenos Aires" },
  "work_conditions": {
    "work_schedule": ["part_time"],
    "work_modality": ["hybrid"],
    "hybrid_onsite_days": 2,
    "employment_type": ["internship"],
    "weekly_hours": { "min": 20, "max": 20 },
    "shifts": ["morning"],
    "availability_start": "within_15_days",
    "salary": {
      "min": 550000,
      "max": 700000,
      "currency": "ARS",
```

```
      "period": "month",
      "gross": true,
      "payment_mode": "local",
      "negotiable": false,
      "adjustment": "quarterly",
      "variable_compensation": [],
      "visibility": "public",
      "to_agree": false
    }
  },
  "requirements": {
    "skills": [
      { "name": "React", "normalized": "react", "level": 2, "kind": "required" },
      { "name": "TypeScript", "normalized": "typescript", "level": 2, "kind": "desired" }
    ],
    "languages": [{ "name": "Inglés", "normalized": "ingles", "level": "B1", "kind": "desired" }],
    "experience": { "min_years": 0, "area": "desarrollo web", "kind": "desired" },
    "academic": { "min_progress_percent": 50, "careers": ["Ing. en Sistemas", "Lic. en Informática"] }
  },
  "benefits": ["días de estudio", "capacitación"],
  "internship": { "institution_agreement": true, "duration_months": 12 }
}
```


Match (extracto):


```
{
  "profile_id": "prof-estudiante-01",
  "job_id": "job-0001",
  "rule_based": {
    "origin": "RULES",
    "engine_version": "0.2.0",
    "score": 82,
    "excluding_cap_applied": false,
    "missing_skills": [
      { "skill": "typescript", "kind": "desired", "required_level": 2, "current_level": 0, "gap": 2, "priority":
"media" }
    ],
    "conditions_fit": {
      "compatible": true,
      "distance_km": 22,
      "checks": [
        { "condition": "work_modality", "result": "match", "profile_value": ["hybrid", "remote"], "job_value":
["hybrid"], "hard": true },
        { "condition": "shifts", "result": "match", "profile_value": ["morning", "afternoon"], "job_value":
["morning"], "hard": false },
        { "condition": "salary", "result": "match", "profile_value": "600000–750000 ARS/mes", "job_value":
"550000–700000 ARS/mes", "hard": false }
      ],
      "salary_fit": {
        "result": "match",
        "expected_monthly": { "min": 600000, "max": 750000 },
        "offered_monthly": { "min": 550000, "max": 700000 },
        "gap_percent": 0,
        "currency": "ARS"
      }
    }
  },
  "ai_recommendations": {
    "origin": "AI",
    "model": "nvidia/…",
    "generated_at": "2026-11-03T12:00:00Z",
    "prompt_version": "training_advisor@1.0.0",
    "disclaimer": "Sugerencia generada por IA; verificá la información.",
    "items": [
      { "type": "course", "skill": "typescript", "text": "Curso introductorio de TypeScript aplicado a React.",
"priority": "media" }
    ]
  },
  "application": { "status": "suggested" }
}
```

9. Impacto estimado si se aprueba (no implementado)

Componente	Cambio
`contracts/`	Nuevo `common_defs.schema.json` (enums y bloques de §3–4) + `student_profile.schema.json`, `job_opportunity.schema.json`, `match.schema.json` con `$ref` a los comunes. `cv_standard` pasaría a `location` como objeto (MAJOR) o se mantiene string y profile-service lo parsea (compatible).
`profile-service`	Entidades `Location`, `WorkConditions`, `AcademicStatus`, `Project`; formulario de preferencias.
`jobs-service`	`WorkConditions`, `benefits`, `internship`, `application`; estados `draft/paused/expired`.
`matching-service`	`ConditionCheck` (Protocol) + una clase por condición; `conditions_fit` en la respuesta; reglas en `scoring_rules.yaml`.
`mcp-tools`	`get_profile` / `get_job` devuelven los bloques nuevos; mocks actualizados.
`n8n` / prompts	`cv_structurer` podría extraer `location` estructurada y preferencias si aparecen en el CV (siempre como sugerencia).
`frontend`	Filtros por modalidad, horario y ubicación; panel "Condiciones" junto a `ScoreCard` (badge verde, reglas).

10. Preguntas abiertas para la daily

1. ¿Condiciones laborales como **filtro separado** (`conditions_fit`, recomendado) o como **criterio con peso** dentro del score?
2. ¿Distancia geográfica real (requiere geocodificación de ciudad o código postal) o sólo comparación ciudad/provincia en el MVP?
3. Sueldo pretendido / ofrecido: ¿qué `visibility` por defecto usamos para cada lado (propuesta: `on_match`)? ¿Hace falta conversión de moneda (`fx_rates`) en el MVP o comparamos sólo con la misma moneda?
4. ¿`location` en `cv_standard` pasa a objeto (rompe contrato) o se mantiene string y se estructura en profile-service?
5. ¿La postulación (`application`) vive en matching-service o en un servicio o módulo aparte?
6. ¿Datos de inclusión (certificado de discapacidad) entran al MVP? Si entran: consentimiento explícito y exclusión total de IA.